

RAMAI - Business Memory Design v1.0

AI Business Operating Assistant for PMI - Documento 05

Scopo. Definire il modello concettuale, logico e operativo della Business Memory, cioè il sistema che consente a RAMAI di ricordare persone, aziende, conversazioni, eventi, opportunità, task e fatti aziendali con provenienza verificabile. Questo documento dettaglia la sezione Business Memory dell'architettura tecnica e diventa il contratto di riferimento per database, pipeline AI, retrieval e Insight Engine.

Principio guida

RAMAI non deve "ricordare" perché un modello linguistico lo ha detto in una conversazione. Deve conoscere un fatto perché esiste una evidenza persistita, tracciabile e rivalutabile.

1. Obiettivi della Business Memory

- Costruire una vista aziendale unificata senza sostituire CRM, email o calendario.
- Rendere interrogabili in linguaggio naturale dati strutturati e contenuti non strutturati.
- Conservare la provenienza di ogni fatto utilizzato dall'AI.
- Distinguere sempre dati osservati, inferenze AI, dati confermati dall'utente e dati obsoleti.
- Consentire correzione, deduplica, merge e revoca delle informazioni.
- Supportare Today, Ask RAMAI, Daily Briefing e generazione di bozze senza hallucination non rilevate.
- Scalare da una singola mailbox a più sorgenti e più utenti mantenendo tenant isolation.

2. Cosa NON è la Business Memory

- Non è la cronologia della chat di Ask RAMAI.
- Non è un vector database in cui salvare tutto indiscriminatamente.
- Non è un CRM duplicato: i sistemi esterni restano system of record quando applicabile.
- Non è un archivio eterno: retention e cancellazione devono rispettare policy e consensi.
- Non trasforma automaticamente una previsione AI in un fatto aziendale.

3. Modello concettuale

La memoria è costruita su quattro livelli. La separazione è importante per evitare che dati raw, fatti verificati e inferenze vengano confusi.

Livello	Contenuto	Esempio	Affidabilità
L0 - Source	Oggetto originale o riferimento al provider	Email Microsoft Graph id=...	Alta come evidenza, non necessariamente come interpretazione
L1 - Event	Evento normalizzato nel dominio RAMAI	EmailReceived, MeetingScheduled	Alta se importato dal provider
L2 - Fact	Affermazione atomica con provenance	Rossi Srl ha un preventivo da 12.500 EUR	Variabile; dipende da source e stato
L3 - Interpretation	Inferenza o raccomandazione	E opportuno fare follow-up oggi	AI; deve restare esplicitamente inferita

Regola: Today e Ask RAMAI possono utilizzare L3 per suggerire, ma quando affermano qualcosa sull'azienda devono poter risalire almeno a un Fact L2 e alla relativa evidenza L0/L1.

4. Entità core v1

Entità	Responsabilità	Campi chiave
Organization	Azienda/ente esterno	Id, TenantId, Name, Domain, LifecycleStatus, ExternalRefs
Person	Persona/contatto	Id, TenantId, DisplayName, Emails, Phones, OrganizationId
Conversation	Thread logico	Provider, ExternalThreadId, Subject, StartedAt, LastActivityAt
Message	Messaggio email/chat importato	ExternalId, ConversationId, Direction, Sender, Recipients, SentAt
Event	Evento normalizzato	Type, OccurredAt, Actor, Subject, SourceRef, Payload
Opportunity	Opportunità commerciale normalizzata	Value, Currency, Stage, Owner, LastActivity, ExternalRef
Task	Attività esplicita o inferita	Type, DueAt, Status, Owner, Confidence, Source
Fact	Fatto atomico interrogabile	Predicate, Subject, Object/Value, ValidFrom/To, Confidence, Status
Relation	Legame tra entità	From, RelationType, To, Validity, Source
Insight	Raccomandazione operativa	Type, Priority, EvidenceRefs, SuggestedAction, Status
MemoryChunk	Contenuto indicizzato semanticamente	EntityScope, Text, Embedding, SourceRefs, Freshness

5. Identita, deduplica e merge

Una delle funzioni più critiche è riconoscere che dati provenienti da sorgenti diverse rappresentano la stessa persona o azienda. Il merge errato è più pericoloso del mancato merge.

5.1 Organization identity

- Chiavi forti: ID CRM, dominio verificato, partita IVA/codice fiscale se presente e lecito.
- Chiavi medie: dominio email aziendale, ragione sociale normalizzata, sito web.
- Chiavi deboli: similarità del nome, firma email, testo conversazionale.
- Merge automatico consentito solo sopra una soglia elevata con almeno una chiave forte o combinazione di segnali indipendenti.
- Merge ambiguo: mantenere entità separate e creare MergeCandidate per futura conferma.

5.2 Person identity

- Email normalizzata e provider contact id sono identificatori primari.
- Alias multipli supportati: una persona può usare più caselle.
- Nome da solo non è sufficiente per il merge automatico.
- Un cambio azienda non deve cancellare la storia precedente: le relazioni Person-Organization hanno validità temporale.

6. Fact model - il cuore della memoria

I Fact sono affermazioni atomiche. Esempi: "Rossi Srl è un cliente", "Mario Rossi lavora per Rossi Srl", "il preventivo X vale 12.500 EUR", "Mario ha chiesto una risposta entro venerdì".

Campo Fact	Descrizione
FactId	Identificatore immutabile
TenantId	Isolamento tenant
SubjectType / SubjectId	Entità a cui il fatto si riferisce
Predicate	Tipo di fatto versionato, es. opportunity.value, contact.role, commitment.due
ObjectEntityId / ValueJson	Entità collegata oppure valore tipizzato
ObservedAt	Quando è stato osservato
ValidFrom / ValidTo	Periodo in cui il fatto è considerato valido
Status	Observed / Inferred / UserConfirmed / Superseded / Rejected
Confidence	0..1; obbligatorio per inferenze, opzionale per provider facts
SourceRef	Riferimento a message/event/provider object
ExtractorVersion	Regola/modello/prompt che ha estratto il fatto
CreatedAt / SupersededBy	Audit e versionamento

I Fact non vengono sovrascritti in-place quando cambia il mondo. Il nuovo fatto supersede il precedente mantenendo la storia.

7. Stati di verita e confidence

Stato	Significato	Uso UI/AI
Observed	Importato direttamente da un system of record o evento verificato	Puo essere presentato come dato, citando la fonte
Inferred	Estratto o dedotto dall AI	Deve essere etichettato e avere confidence + evidenza
UserConfirmed	Confermato esplicitamente da un utente autorizzato	Prevalenza sulle inferenze, salvo nuova evidenza forte
Superseded	Sostituito da un fatto piu recente	Non usato come corrente; resta nella timeline
Rejected	Smentito dall utente o da regola deterministica	Non deve essere riproposto senza nuova evidenza

7.1 Confidence policy

- Confidence non e probabilita matematica universale: e un segnale operativo calibrato per extractor e fact type.
- Soglia alta: fatto inferito puo alimentare ranking e suggerimenti, ma resta marcato come inferenza.
- Soglia media: puo apparire come "possibile" e richiedere conferma.
- Soglia bassa: conservare eventualmente come candidate, non esporre come informazione aziendale.
- La soglia deve essere configurabile per Predicate, non globale.

8. Provenance model

Ogni informazione importante deve poter rispondere a: Da dove viene? Quando l abbiamo vista? Chi o cosa l ha interpretata? Quale versione del modello/regola e stata usata?

Source type	Reference minima
Email/Message	Provider, mailbox connection, external message id, timestamp, participant refs
CRM object	Provider, object type, external id, last modified, sync cursor
Calendar event	Provider, event id, organizer, start/end, last modified
User input	UserId, timestamp, screen/action, previous value se modifica
AI extraction	Model/provider, prompt/extractor version, input source refs, structured output hash
Rule engine	Rule id/version, evaluated facts/events, execution id

8.1 Evidence bundle

Un Insight deve salvare un EvidenceBundle immutabile contenente i SourceRef e FactId utilizzati al momento della generazione. In questo modo un insight storico resta spiegabile anche se la memoria corrente cambia.

9. Event model e timeline

Gli Event rappresentano cio che e accaduto e alimentano sia la timeline sia le pipeline. Gli eventi sono append-only; eventuali correzioni generano eventi successivi.

Categoria	Eventi MVP
Communication	EmailReceived, EmailSent, ThreadUpdated
Calendar	MeetingScheduled, MeetingUpdated, MeetingCancelled
CRM	OpportunityCreated, OpportunityChanged, ContactUpdated
Task	TaskCreated, TaskCompleted, TaskDeferred
Memory	FactCreated, FactSuperseded, FactConfirmed, FactRejected, EntitiesMerged
Insight	InsightCreated, InsightOpened, InsightDone, InsightIgnored, InsightSnoozed
Connector	SyncStarted, SyncCompleted, SyncFailed, ConnectionRevoked

9.1 Timeline projection

La timeline visualizzata nella scheda Business Memory non deve interrogare provider esterni in tempo reale. E una proiezione locale costruita da Event e Message normalizzati, ordinata per OccurredAt, con link alla fonte originale quando disponibile.

10. Relationship graph

Non serve un graph database dedicato nell MVP. Le relazioni possono essere memorizzate in PostgreSQL con una tabella Relation tipizzata e indici appropriati; il modello deve pero essere concettualmente a grafo.

RelationType	Esempio
WORKS_FOR	Person -> Organization
CONTACT_OF	Person -> Organization / Opportunity
PARTICIPATED_IN	Person -> Conversation / Meeting
OWNS	User/Person -> Opportunity / Task
ABOUT	Conversation/Message -> Organization / Opportunity
MENTIONS	Message -> Person/Organization
DERIVED_FROM	Fact/Insight -> Source/Event/Fact
RELATED_TO	Entita -> Entita, solo se semanticamente definita e non generica

Relazioni inferite devono avere la stessa disciplina dei Fact: source, confidence, status e validita temporale.

11. MemoryChunk ed embeddings

Gli embeddings sono un indice di recupero, non la memoria autorevole. Il testo indicizzato deve essere derivato da oggetti già normalizzati e deve mantenere riferimenti alle fonti.

Campo	Regola
ChunkId / TenantId	Sempre tenant scoped
ScopeType / ScopeId	Organization, Person, Conversation, Message, Opportunity o Fact
Text	Testo normalizzato; evitare duplicati e boilerplate
Embedding	Vector generato con modello/versione registrati
SourceRefs	Una o piu fonti verificabili
ContentHash	Evita re-embedding quando il contenuto non cambia
FreshnessAt	Ultima data significativa del contenuto
SensitivityClass	Categoria utile a policy di retrieval/retention
EmbeddingVersion	Consente migrazione o re-indexing controllato

11.1 Chunking policy

- Email: chunk per singolo messaggio pulito + opzionale summary di thread separato.
- Conversation: non concatenare thread enormi senza segmentazione temporale.
- Opportunity: snapshot testuale generato da campi strutturati, con hash.
- Fact: embedding solo per predicate/valori utili alla ricerca; evitare embedding di semplici ID.
- Contenuti sensibili esclusi o minimizzati secondo policy futura Security/GDPR.

12. Retrieval per Ask RAMAI

Ask RAMAI deve usare retrieval ibrido: filtri strutturati + query relazionali + full text/vector search. Il modello non decide autonomamente quale tenant o quali dati puo vedere.

Query pipeline

1. Autenticazione + TenantContext
2. Intent classification
3. Entity resolution (Rossi, preventivo, periodo)
4. Query strutturata su Facts/Entities/Events
5. Retrieval semantico solo se utile
6. Evidence selection e freshness checks
7. Generazione risposta con citazioni interne
8. Validation: nessuna entita/importo senza evidence
9. Audit della query e delle fonti utilizzate

Per domande come "quali clienti non sentiamo da 30 giorni?" la risposta deve derivare prima da query temporali strutturate, non da similarita vettoriale.

13. Memory update pipeline

Pipeline standard per ogni oggetto sincronizzato:

```
Provider Object -> Adapter -> Normalize -> Idempotency/Dedup -> Entity Resolution -> Event -> Deterministic Facts  
-> AI Extraction Candidates -> Validation -> Facts/Relations -> Embedding/Index -> Insight Candidates
```

13.1 Deterministic first

- Campi espliciti del provider devono essere mappati deterministicamente quando possibile.
- Usare AI solo per ciò che richiede comprensione semantica: intent, commitment, relazione implicita, topic, riassunto.
- Non chiedere al modello di indovinare valori che il provider già espone.
- Le estrazioni AI devono usare structured output e schema validato.

13.2 Idempotency

- Ogni source object ha ExternalRef univoco per Tenant + Connection + Provider.
- Stesso evento importato più volte non crea duplicati.
- Cambio del contenuto genera nuova versione/evento e rivalutazione dei Fact dipendenti.
- Job retry-safe: ogni stage deve poter essere rieseguito.

14. Contraddizioni e aggiornamento dei fatti

Quando emergono due fatti incompatibili, RAMAI non deve scegliere silenziosamente il più comodo. Deve applicare regole di precedence e mantenere audit.

Caso	Comportamento
CRM dice valore 10k, email dice 12k	Se CRM è system of record: current=10k; email può generare candidate/alert
Utente corregge ruolo contatto	UserConfirmed prevale sull'inferenza precedente
Nuova email indica cambio azienda	Crea nuova relazione con ValidFrom; non distrugge la precedente
Due inferenze discordanti	Mantieni candidate separate; non promuovere a current fact senza soglia/evidenza
Fonte cancellata/revocata	Applica retention policy; facts dipendenti diventano unavailable/retracted se richiesto

15. Freshness e decadimento

Non tutti i fatti restano utili per sempre. Ogni Predicate deve poter definire una FreshnessPolicy.

Tipo	Esempio policy
Identità stabile	Nome legale: no decay automatico, ma supersedibile
Ruolo persona	Rivalutare con nuove firme/email/CRM; warning se molto vecchio
Opportunity stage	Deve provenire dall'ultimo snapshot/evento CRM
Commitment	Valido fino a completamento/scadenza/revoca
Sentiment/topic	Context-only, decay rapido; mai fact forte
Last contact	Calcolato dinamicamente dagli eventi, non salvato come verità permanente

16. Insight Engine: contratto con la memoria

L Insight Engine non deve leggere direttamente testo raw senza passare dal Memory API salvo casi esplicitamente previsti. Deve ricevere CandidateContext composto da entita, fact, eventi e evidenze.

Insight	Prerequisiti minimi
Email senza risposta	Incoming request/commitment + assenza di outgoing response successiva + source refs
Follow-up preventivo	Opportunity/open quote + last meaningful contact + nessun task futuro equivalente
Lead fermo	Lifecycle/Opportunity stage + inactivity window + owner/context
Meeting prep	Evento calendar imminente + organizzazione/persona risolta + contesto recente
Cliente dimenticato	Relationship/customer + last meaningful interaction oltre soglia configurata

16.1 Evidence-first Insight

Ogni Insight persistito deve contenere: InsightType, SubjectRef, Priority, ReasonCode, EvidenceBundle, SuggestedAction, GeneratedAt, GeneratorVersion, Expiry/Freshness, Status. La motivazione UI deve essere ricostruibile senza chiamare nuovamente il modello.

17. Correzioni dell utente e learning loop

- Correggi dato: crea Fact UserConfirmed o Relationship correction; non modifica raw source.
- Insight Non utile: salva feedback legato a InsightType, ReasonCode e contesto, senza trasformarlo automaticamente in training data.
- Merge/unmerge entita: operazione auditata e reversibile quando tecnicamente possibile.
- Ignora suggerimento: modifica l insight, non nega necessariamente il fatto sottostante.
- Regole future di personalizzazione devono usare feedback aggregato/tenant-specific con limiti espliciti.

18. Data model PostgreSQL - baseline

Schema logico minimo; i nomi definitivi saranno consolidati durante lo Sprint 4.

```
Organizations
People
PersonOrganizations
Conversations
Messages
Events
Opportunities
Tasks
Facts
Relations
MemoryChunks (vector)
Insights
InsightEvidence
ExternalReferences
EntityMergeCandidates
MemoryCorrections
AIExtractions / AIUsage
```

18.1 Indici obbligatori

- TenantId + Id su tutte le tabelle tenant-scoped.
- TenantId + ExternalRef(provider, connection, objectType, externalId) unique.
- TenantId + Organization normalized domain/name lookup.
- TenantId + Person email normalized.
- TenantId + Event OccurredAt.
- TenantId + Fact Subject + Predicate + Status + ValidTo.
- TenantId + Insight Status + Priority + CreatedAt.
- pgvector index partition/query constrained sempre per TenantId.

19. API/application contracts

Servizio	Responsabilita
EntityResolver	Find/create/merge candidate per Person e Organization
MemoryWriter	Applica Event, Fact, Relation e versioning in transazione coerente
MemoryReader	Vista corrente + timeline + provenance
EvidenceService	Costruisce EvidenceBundle da Facts/Events/Sources
SemanticIndex	Upsert/delete/search MemoryChunk tenant-scoped
FactExtractor	Estrazione AI structured-output; non scrive direttamente nel DB
FactValidator	Schema, confidence policy, contradictions, source presence
MemoryProjection	Costruisce viste aggregate per UI/Today/Ask
CorrectionService	Conferma/rifiuta/corregge/merge con audit

20. Tenant isolation e security

- TenantId non viene mai accettato come autorità da parametri client: deriva dal contesto autenticato.
- Qualsiasi query vettoriale include filtro tenant obbligatorio.
- Source refs e external ids non devono consentire IDOR verso oggetti di altri tenant.
- Contenuto raw non necessario alla UX può essere minimizzato; allegati non importati per default nell MVP.
- Accesso a mailbox condivise e dati team deve rispettare membership/ruoli futuri.
- Audit separato per letture sensibili e modifiche della memoria dove richiesto.

21. Retention, delete e revoca

La strategia dettagliata verrà finalizzata nel documento Security/GDPR, ma la Business Memory deve già supportare la cancellazione propagata.

- Disconnect non equivale automaticamente a delete: la UI deve distinguere chiaramente le due azioni.
- Delete source/tenant deve poter rimuovere raw data, chunks, embeddings, facts derivati e source references secondo policy.
- I job di delete devono essere idempotenti e auditati.
- Backup retention e tempi di cancellazione devono essere documentati.
- I contenuti cancellati non devono riapparire tramite cache o vector index.

22. Osservabilità e metriche

Metrica	Scopo
entity_resolution_auto_merge_rate	Qualità risoluzione identità
merge_correction_rate	Rilevare merge automatici errati
fact_extraction_accept_rate	Qualità extractor per predicate
fact_rejection_rate	Misurare errori/inferenze non affidabili
provenance_missing_rate	Deve tendere a zero per facts usati dagli insight
memory_freshness_lag	Ritardo tra provider change e memoria corrente
retrieval_evidence_hit_rate	Risposte Ask con evidenze sufficienti
insight_evidence_valid_rate	Insight con bundle completo e ancora valido

23. Test strategy specifica Business Memory

Test	Esempi
Unit	fact supersede, validity, precedence, confidence policy, relation rules
Property-based	idempotency, merge/unmerge invariants, no duplicate ExternalRef
Integration	PostgreSQL + pgvector, transactions, indexes, concurrent sync
Tenant security	cross-tenant entity lookup, vector retrieval, source ref leakage
AI eval	commitment extraction, organization resolution, role extraction, hallucination rate
Replay	payload email/CRM anonimizzati riprocessati con nuove extractor version
E2E	email -> entity -> fact -> insight -> Ask response -> evidence view

24. Golden dataset per lo sviluppo

Prima della beta va creato un piccolo dataset sintetico e anonimizzato con almeno 100-200 conversazioni rappresentative. Deve contenere casi facili e ambigui: omonimi, alias email, cambio azienda, richieste implicite, preventivi, follow-up, risposte tardive, thread lunghi, firme, newsletter e messaggi automatici.

- Ogni esempio ha expected entities, expected facts e fatti che NON devono essere estratti.
- Versionare il dataset nel repository o in storage di test protetto.
- Eseguire eval automatiche ad ogni modifica di prompt/modello/extractor.
- Bloccare release se peggiorano metriche di precisione su fact ad alto impatto.

25. MVP scope vs futuro

MVP	Futuro
Email Microsoft 365, Calendar essenziale	Gmail, CRM multipli, WhatsApp/VoIP
Organization/Person/Conversation/Message/Event/Fact	Document, Invoice, Product, Contract, Project
PostgreSQL relation graph	Graph DB solo se query reali lo giustificano
pgvector	Hybrid search avanzato / reranker dedicato
User correction semplice	Policy engine e governance enterprise
Single/limited team context	Fine-grained ACL per business unit / data domain

26. Sprint di implementazione della Business Memory

Sprint	Deliverable
BM-0	Predicate catalog, schemas, conventions, seed test data
BM-1	Organization/Person + ExternalReference + EntityResolver
BM-2	Conversation/Message/Event + timeline projection
BM-3	Fact/Relation store + provenance + supersede model
BM-4	AI extraction candidates + validator + confidence policy
BM-5	MemoryChunk + embeddings + hybrid retrieval
BM-6	MemoryReader API + Business Memory UI contracts
BM-7	EvidenceBundle + Insight integration
BM-8	Corrections/merge candidates + audit
BM-9	Eval suite, tenant security tests, performance baseline

27. Predicate catalog iniziale

I Predicate devono essere un catalogo controllato/versionato, non stringhe inventate liberamente dal modello.

Area	Predicate esempi
Identity	person.role, person.organization, organization.lifecycle_status
Commercial	opportunity.value, opportunity.stage, opportunity.expected_close
Communication	conversation.topic, communication.requested_action, communication.reply_expected
Commitment	commitment.action, commitment.owner, commitment.due_at, commitment.status
Relationship	relationship.customer_since, relationship.primary_contact
Operational	task.type, task.due_at, task.status
Classification	message.category, message.is_automated, message.is_marketing

28. Acceptance criteria v1.0

- Ogni Fact utilizzato da Today/Ask ha almeno un SourceRef valido.
- Ogni Fact inferito contiene Confidence, ExtractorVersion e stato Inferred/UserConfirmed.
- Un sync ripetuto dello stesso oggetto non crea duplicati.
- Una correzione utente non modifica il payload sorgente e rimane auditabile.
- Un Fact aggiornato conserva la versione precedente come Superseded.
- La timeline mostra eventi in ordine coerente e linka le fonti disponibili.
- Ask RAMAI non restituisce un importo, data o persona specifica senza evidence recuperata.
- La ricerca semantic/vector non puo restituire dati cross-tenant.
- Eliminazione di una source viene propagata agli indici derivati secondo policy.
- I principali extractor superano le soglie definite sul golden dataset prima della beta.

29. Definition of Business Memory Ready

- Schema DB e migrations approvate.
- Predicate catalog v1 versionato.
- EntityResolver e ExternalReference idempotenti.
- Fact/Relation/Event store con provenance e versioning.
- MemoryChunk + pgvector tenant-safe.
- EvidenceBundle utilizzabile da Insight Engine e Ask RAMAI.
- Correzioni/merge candidate disponibili nell API.
- Eval AI e security test automatizzati in CI.
- Metriche operative espote in observability.
- Documentazione aggiornata per handoff a Codex.

30. Decisioni aperte da chiudere prima del coding BM-3

- Periodo import email iniziale default (es. 30/60/90 giorni) e limiti Free.
- Quali contenuti del body email salvare integralmente e quali normalizzare/minimizzare.
- Catalogo preciso dei Fact consentiti nell MVP e relative soglie confidence.
- Provider/modello embeddings iniziale e dimensione vector.
- System of record precedence quando saranno aggiunti i CRM.
- Policy di retention, export e delete da consolidare nel documento 08 Security/GDPR.

Versione: 1.0 - 29 settembre 2026

Documento precedente: 04_Technical_Architecture_v1.0

Documento successivo previsto: 06_AI_Engine_Design_v1.0